

Lesson 14 Notes: Modular Menu-Driven Matrix Program

1. Main idea of Lesson 14

Lesson 14 does not introduce a new matrix type.

Before this lesson, you already learned how to store special matrices using a 1D array. You also learned how to write a menu-driven program in Lesson 13.

Lesson 14 improves the Lesson 13 menu program by making it **modular**.

A modular program is a program split into smaller functions. Each function has one clear job.

In Lesson 13, the menu cases may have contained all the logic directly inside the `switch` statement.

Example:

```
switch (choice) {  
    case 1:  
        // create logic here  
        break;  
  
    case 2:  
        // get logic here  
        break;  
  
    case 3:  
        // set logic here  
        break;  
  
    case 4:  
        // display logic here  
        break;  
}
```

This works, but it can become messy when the program grows.

Lesson 14 teaches this cleaner style:

```
switch (choice) {  
    case 1:
```

```

        create(A, n);
        break;

    case 2:
        cout << get(A, i, j);
        break;

    case 3:
        set(A, i, j, x);
        break;

    case 4:
        display(A, n);
        break;

}

```

Now the menu only controls which operation runs. The actual work is done inside separate helper functions.

2. Current progress before Lesson 14

Before Lesson 14, the course has already covered the main matrix storage ideas.

Lesson range	What was learned	Main idea
Lessons 1–4	Diagonal matrices	Store only values where <code>i == j</code> .
Lessons 5–8	Lower triangular matrices	Store values where <code>i >= j</code> .
Lesson 9	Upper triangular matrices	Store values where <code>i <= j</code> .
Lesson 10	Symmetric matrices	Store one triangular half and use the mirror rule.
Lesson 11	Tridiagonal and band matrices	Store values near the main diagonal using <code>i - j</code> .
Lesson 12	Toeplitz matrices	Store repeated diagonals using the first row and first column.
Lesson 13	Menu-driven matrix program	Let the user choose create, get, set, display, or exit.
Lesson 14	Modular menu-driven program	Split the menu program into helper functions.

So Lesson 14 is not about a new formula.

It is about **better program organization**.

3. The recurring pattern in special matrix programs

All special matrix programs follow the same basic pattern:

1. Decide whether matrix position `(i, j)` is meaningful.
2. Store only meaningful values in a 1D array.
3. Use a direct formula to convert `(i, j)` into an array index.
4. For `get()`, return the stored value if the position is meaningful.
5. For `get()`, return `0` if the position is a structural zero.
6. For `set()`, store the value only if the position is valid for that matrix type.
7. For `display()`, print the full matrix using real stored values and fake zeros.

Lesson 14 uses the same idea, but it separates these operations into functions.

4. What is a menu-driven program?

A menu-driven program repeatedly shows choices to the user.

Example:

```
Menu
1. Create matrix
2. Get element
3. Set element
4. Display matrix
0. Exit
Enter choice:
```

The user enters a number.

The program checks the number and runs the matching operation.

Choice	Operation
1	Create or fill the matrix.
2	Get one element.
3	Set or update one element.
4	Display the full matrix.

Choice	Operation
0	Exit the program.

A menu-driven program usually uses:

```
do {
    // show menu
    // read choice
    // perform operation
} while (choice != 0);
```

A `do-while` loop is useful because the menu should appear at least once.

5. What does modular mean?

Modular means the program is divided into smaller parts.

Each part has one clear job.

In Lesson 14, the main helper functions are:

Function	Job
<code>create()</code>	Fill the stored 1D array.
<code>get()</code>	Return one matrix value.
<code>set()</code>	Change one matrix value if the position is valid.
<code>display()</code>	Print the full visible matrix.
<code>main()</code>	Control the menu and call the correct function.

Simple idea:

```
main() controls the program.
create() fills the matrix.
get() reads a value.
set() changes a value.
display() prints the matrix.
```

So `main()` should not do every detail.

`main()` should behave like a manager.

The helper functions should behave like workers.

6. Why modular programming is useful

Modular programming is useful because it makes the program easier to read, test, and modify.

6.1 Easier to read

Without functions, the menu cases become long and crowded.

Example of messy style:

```
case 4:
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (i == j) {
                cout << A[i - 1] << " ";
            } else {
                cout << "0 ";
            }
        }
        cout << endl;
    }
    break;
```

With a function, the menu case is much cleaner:

```
case 4:
    display(A, n);
    break;
```

Now the menu is easier to understand.

6.2 Easier to test

If `get()` is wrong, you only inspect the `get()` function.

If `display()` is wrong, you only inspect the `display()` function.

You do not need to search through a long `switch` statement.

6.3 Easier to modify

If you want to change from a diagonal matrix to a lower triangular matrix, you mostly change:

1. The array size.
2. The storage condition.
3. The index formula.
4. The helper functions.

The menu structure can stay almost the same.

7. What belongs in `main()`?

In Lesson 14, `main()` should control the program.

It should not contain all the matrix logic.

The main responsibilities of `main()` are:

1. Declare the main variables.
2. Read the matrix dimension.
3. Allocate memory for the 1D array.
4. Show the menu repeatedly.
5. Read the user's choice.
6. Use `switch-case` to call the correct helper function.
7. Release dynamic memory before the program ends.

For a diagonal matrix example, `main()` needs variables like this:

```
int n;  
int choice;  
int i, j, x;  
int *A;
```

Meaning:

Variable	Meaning
<code>n</code>	Matrix dimension.
<code>choice</code>	User menu choice.
<code>i</code>	Row number.
<code>j</code>	Column number.

Variable	Meaning
x	Value to store.
A	Pointer to the 1D storage array.

8. Diagonal matrix reminder

Lesson 14 often uses a diagonal matrix as the example because it is the simplest special matrix.

A diagonal matrix stores only values on the main diagonal.

The condition is:

```
i == j
```

That means:

Position type	Condition	What happens?
Diagonal position	<code>i == j</code>	Store or return the value.
Non-diagonal position	<code>i != j</code>	Treat as zero.

Example diagonal matrix:

```
5 0 0
0 8 0
0 0 9
```

Stored array:

```
A = [5, 8, 9]
```

Mapping:

```
M[1][1] -> A[0]
M[2][2] -> A[1]
M[3][3] -> A[2]
```

Index formula:

```
index = i - 1
```

This formula is used only when:

```
i == j
```

9. Structural zeros

A structural zero is a zero that exists because of the matrix rule.

In a diagonal matrix, every non-diagonal position is a structural zero.

Example:

```
M[1][3] = 0
```

Why?

Because:

```
1 != 3
```

So `(1, 3)` is not on the main diagonal.

The value is not stored in the array.

The program simply returns or prints `0` when needed.

This is why special matrix storage saves memory.

10. Matrix indexing vs array indexing

In these lessons, matrix positions usually use 1-based indexing.

That means matrix rows and columns start at `1`.

Examples:


```
M[1][1]
M[2][2]
M[3][3]
```

But C++ arrays use 0-based indexing.

That means array indexes start at 0.

Examples:

```
A[0]
A[1]
A[2]
```

So we subtract 1 when converting a matrix position to an array index.

Example:

```
M[1][1] -> A[1 - 1] -> A[0]
M[2][2] -> A[2 - 1] -> A[1]
M[3][3] -> A[3 - 1] -> A[2]
```

That is why the formula is:

```
A[i - 1]
```

11. Dynamic memory in Lesson 14

The matrix values are stored in a dynamic array.

For a diagonal matrix of size `n`, we store only `n` values.

So we allocate:

```
int *A = new int[n]{};
```

Meaning:

Code part	Meaning
<code>int *A</code>	<code>A</code> is a pointer to an integer array.
<code>new int[n]</code>	Create an array of <code>n</code> integers on the heap.
<code>{}</code>	Initialize the array values to zero.

The `{}` is useful because it prevents garbage values.

At the end of the program, we release the memory:

```
delete[] A;
```

Important rule:

```
new[] must be matched with delete[]
```

Correct:

```
int *A = new int[n]{};
delete[] A;
```

Wrong:

```
int *A = new int[n]{};
delete A;
```

Use `delete[]` because `A` points to an array, not one single integer.

12. Function prototypes

A function prototype tells C++ that a function exists.

It is written before `main()`.

Example:

```
void create(int *A, int n);
int get(int *A, int i, int j);
```

```
void set(int *A, int i, int j, int x);
void display(int *A, int n);
```

Why do we need prototypes?

Because `main()` may call these functions before the compiler has seen their full definitions.

The prototype gives the compiler the function name, return type, and parameter list.

Prototype	Meaning
<code>void create(int *A, int n);</code>	A function that fills the array and returns nothing.
<code>int get(int *A, int i, int j);</code>	A function that returns one integer value.
<code>void set(int *A, int i, int j, int x);</code>	A function that updates the array and returns nothing.
<code>void display(int *A, int n);</code>	A function that prints the matrix and returns nothing.

13. Why do we pass `int *A` to functions?

`A` is a pointer to the stored array.

When we pass `A` into a function, we pass the address of the array.

We do not copy the whole array.

Example:

```
set(A, 2, 2, 5);
```

Inside `set()`, the function receives the same array address.

So if `set()` changes an array value, the change remains after the function ends.

Simple memory picture:

```
main has A ---> [5, 8, 9]

set() receives the same address.
```

```
set() also sees:  
A ---> [5, 8, 9]
```

If `set()` changes `A[2]`, the real array changes.

Example:

```
set(A, 3, 3, 10);
```

Before:

```
A = [5, 8, 9]
```

After:

```
A = [5, 8, 10]
```

This works because `A` is a pointer.

14. Helper function 1: `create()`

The `create()` function fills the stored array.

For a diagonal matrix, it reads only diagonal elements.

Code:

```
void create(int *A, int n) {  
    cout << "Enter diagonal elements:\n";  
  
    for (int i = 1; i <= n; i++) {  
        cin >> A[i - 1];  
    }  
}
```

Explanation

The loop runs from `1` to `n` because matrix positions are written using 1-based indexing.

For each `i`, it stores the value at:

A[i - 1]

because C++ array indexes start at 0.

Dry run of create()

Suppose:

n = 3

Input:

5 8 9

Loop trace:

i	Input	Stored at	Array after storing
1	5	A[0]	[5, 0, 0]
2	8	A[1]	[5, 8, 0]
3	9	A[2]	[5, 8, 9]

Final stored array:

A = [5, 8, 9]

This represents the full matrix:

```
5 0 0
0 8 0
0 0 9
```

15. Helper function 2: get()

The get() function returns the value at a matrix position.

Code:

```
int get(int *A, int i, int j) {  
    if (i == j) {  
        return A[i - 1];  
    }  
  
    return 0;  
}
```

Explanation

For a diagonal matrix:

- If `i == j`, the position is diagonal.
- If `i != j`, the position is a structural zero.

So `get()` checks:

```
if (i == j)
```

If true, it returns the stored value:

```
A[i - 1]
```

If false, it returns:

```
0
```

Dry run 1: Getting a diagonal value

Call:

```
get(A, 2, 2);
```

Assume:

```
A = [5, 8, 9]
```

Step-by-step:

```
i = 2  
j = 2
```

Check:

```
2 == 2
```

This is true.

Calculate index:

```
index = i - 1  
index = 2 - 1  
index = 1
```

Return:

```
A[1] = 8
```

Answer:

```
8
```

Dry run 2: Getting a non-diagonal value

Call:

```
get(A, 1, 3);
```

Step-by-step:

```
i = 1  
j = 3
```

Check:

```
1 == 3
```

This is false.

So the function returns:

0

The function does not check the array because it already knows (1, 3) is a structural zero.

16. Helper function 3: set()

The set() function changes a matrix value.

For a diagonal matrix, it only changes diagonal positions.

Code:

```
void set(int *A, int i, int j, int x) {
    if (i == j) {
        A[i - 1] = x;
        cout << "Element updated.\n";
    } else {
        cout << "Cannot set non-diagonal position in a diagonal matrix.\n";
    }
}
```

Explanation

The function receives:

Parameter	Meaning
A	Pointer to the stored array.
i	Row number.
j	Column number.
x	New value to store.

The function checks:

```
if (i == j)
```


If true, it stores the value:

```
A[i - 1] = x;
```

If false, it refuses to store the value because non-diagonal positions must stay zero.

Dry run 1: Setting a diagonal value

Call:

```
set(A, 3, 3, 10);
```

Assume:

```
A = [5, 8, 9]
```

Step-by-step:

```
i = 3  
j = 3  
x = 10
```

Check:

```
3 == 3
```

This is true.

Calculate index:

```
index = i - 1  
index = 3 - 1  
index = 2
```

Store:

```
A[2] = 10
```

Array after update:

```
A = [5, 8, 10]
```

Dry run 2: Trying to set a non-diagonal value

Call:

```
set(A, 1, 3, 99);
```

Step-by-step:

```
i = 1  
j = 3  
x = 99
```

Check:

```
1 == 3
```

This is false.

So the function does not store `99`.

Why?

Because `(1, 3)` is not a diagonal position.

In a diagonal matrix, `(1, 3)` must remain zero.

17. Helper function 4: `display()`

The `display()` function prints the full visible matrix.

Code:

```
void display(int *A, int n) {  
    for (int i = 1; i <= n; i++) {  
        for (int j = 1; j <= n; j++) {  
            if (i == j) {  
                cout << A[i - 1] << " ";  
            }  
        }  
    }  
}
```

```

        } else {
            cout << "0 ";
        }
    }
    cout << endl;
}
}

```

Explanation

The stored array contains only diagonal values.

Example:

```
A = [5, 8, 10]
```

But the visible matrix is a full square matrix:

```

5 0 0
0 8 0
0 0 10

```

So `display()` uses two loops:

Loop	Purpose
Outer loop	Controls rows.
Inner loop	Controls columns.

For each position `(i, j)`, it checks:

```
if (i == j)
```

If true, it prints the stored diagonal value.

If false, it prints `0`.

Dry run of `display()`

Assume:

n = 3
A = [5, 8, 10]

The function checks every visible position.

Row 1:

Position	Check	Output
(1,1)	1 == 1	A[0] = 5
(1,2)	1 != 2	0
(1,3)	1 != 3	0

Output row:

5 0 0

Row 2:

Position	Check	Output
(2,1)	2 != 1	0
(2,2)	2 == 2	A[1] = 8
(2,3)	2 != 3	0

Output row:

0 8 0

Row 3:

Position	Check	Output
(3,1)	3 != 1	0
(3,2)	3 != 2	0
(3,3)	3 == 3	A[2] = 10

Output row:

```
0 0 10
```

Full output:

```
5 0 0
0 8 0
0 0 10
```

18. Full modular diagonal matrix program

This is the complete modular version of the diagonal matrix menu program.

```
#include <iostream>
using namespace std;

// Function prototypes
void create(int *A, int n);
int get(int *A, int i, int j);
void set(int *A, int i, int j, int x);
void display(int *A, int n);

int main() {
    int n;
    int choice;
    int i, j, x;

    cout << "Enter matrix dimension: ";
    cin >> n;

    // A diagonal matrix stores only n values.
    int *A = new int[n]{};

    do {
        cout << "\nMenu\n";
        cout << "1. Create diagonal matrix\n";
        cout << "2. Get element\n";
        cout << "3. Set element\n";
        cout << "4. Display matrix\n";
        cout << "0. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;
```

```

        switch (choice) {
            case 1:
                create(A, n);
                break;

            case 2:
                cout << "Enter row and column: ";
                cin >> i >> j;
                cout << "Element is " << get(A, i, j) << endl;
                break;

            case 3:
                cout << "Enter row, column, and new value: ";
                cin >> i >> j >> x;
                set(A, i, j, x);
                break;

            case 4:
                display(A, n);
                break;

            case 0:
                cout << "Exiting program.\n";
                break;

            default:
                cout << "Invalid choice.\n";
        }

    } while (choice != 0);

    delete[] A;
    return 0;
}

void create(int *A, int n) {
    cout << "Enter diagonal elements:\n";

    for (int i = 1; i <= n; i++) {
        cin >> A[i - 1];
    }
}

int get(int *A, int i, int j) {
    if (i == j) {
        return A[i - 1];
    }
}

```

```

        return 0;
    }

    void set(int *A, int i, int j, int x) {
        if (i == j) {
            A[i - 1] = x;
            cout << "Element updated.\n";
        } else {
            cout << "Cannot set non-diagonal position in a diagonal matrix.\n";
        }
    }

    void display(int *A, int n) {
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                if (i == j) {
                    cout << A[i - 1] << " ";
                } else {
                    cout << "0 ";
                }
            }
            cout << endl;
        }
    }
}

```

19. Full program dry run

Suppose the user enters:

n = 3

The program creates:

```
int *A = new int[3]{};
```

Initial array:

A = [0, 0, 0]

Step 1: User chooses option 1

Menu choice:

1

The program calls:

```
create(A, n);
```

Input:

5 8 9

Array becomes:

```
A = [5, 8, 9]
```

Visible matrix represented:

```
5 0 0
0 8 0
0 0 9
```

Step 2: User chooses option 2

Menu choice:

2

Input row and column:

2 2

The program calls:

```
get(A, 2, 2);
```


Check:

```
2 == 2
```

Return:

```
A[1] = 8
```

Output:

```
Element is 8
```

Step 3: User chooses option 3

Menu choice:

```
3
```

Input:

```
3 3 10
```

The program calls:

```
set(A, 3, 3, 10);
```

Check:

```
3 == 3
```

Index:

```
3 - 1 = 2
```

Update:

```
A[2] = 10
```

Array becomes:

```
A = [5, 8, 10]
```

Step 4: User chooses option 4

Menu choice:

```
4
```

The program calls:

```
display(A, n);
```

Output:

```
5 0 0  
0 8 0  
0 0 10
```

Step 5: User chooses option 0

Menu choice:

```
0
```

The program exits the loop.

Then it runs:

```
delete[] A;
```

This releases the heap memory.

20. Pointer passing dry run

This is one of the most important ideas in Lesson 14.

Suppose `main()` has this array:

```
A = [5, 8, 9]
```

The array is stored on the heap.

`A` stores the address of the first element.

Now `main()` calls:

```
set(A, 3, 3, 10);
```

Inside the function:

```
void set(int *A, int i, int j, int x)
```

The parameter `A` receives the same address.

So both `main()` and `set()` point to the same heap array.

Memory picture:

```
main's A  -----  
  
              ----> [5, 8, 9]  
              /  
set's A    -----
```

When `set()` does:

```
A[2] = 10;
```

The actual heap array changes.

After the function ends, `main()` still sees:

```
A = [5, 8, 10]
```

That is why passing `int *A` is enough to update the array.

21. Difference between menu logic and matrix logic

Lesson 14 separates two types of logic.

21.1 Menu logic

Menu logic means:

- Show choices.
- Read the choice.
- Decide which operation to call.
- Repeat until the user exits.

This belongs in `main()`.

Example:

```
switch (choice) {  
    case 1:  
        create(A, n);  
        break;  
  
    case 2:  
        cout << get(A, i, j);  
        break;  
  
    case 3:  
        set(A, i, j, x);  
        break;  
  
    case 4:  
        display(A, n);  
        break;  
}
```

21.2 Matrix logic

Matrix logic means:

- Check whether a position is stored.
- Calculate the array index.
- Store a value.
- Return a value.
- Print the matrix.

This belongs in helper functions.

Example:

```
int get(int *A, int i, int j) {  
    if (i == j) {  
        return A[i - 1];  
    }  
  
    return 0;  
}
```

This separation makes the program cleaner.

22. How Lesson 14 connects to other matrix types

Lesson 14 uses a diagonal matrix example, but the same modular idea works for other special matrices.

The menu can stay almost the same.

Only the helper functions and array size need to change.

22.1 Diagonal matrix version

Stored condition:

```
i == j
```

Array size:

```
n
```

Index formula:

$$i - 1$$

22.2 Lower triangular matrix version

Stored condition:

$$i \geq j$$

Array size:

$$n * (n + 1) / 2$$

Row-major index formula:

$$i * (i - 1) / 2 + (j - 1)$$

22.3 Upper triangular matrix version

Stored condition:

$$i \leq j$$

Array size:

$$n * (n + 1) / 2$$

The index formula depends on whether row-major or column-major storage is used.

22.4 Main lesson

The structure stays the same:

```
create(A, n);
get(A, i, j);
set(A, i, j, x);
display(A, n);
```

But the formulas inside the helper functions change depending on the matrix type.

This is called the **formula-swapping idea**.

23. Complexity analysis

Complexity means how much time or memory the program uses as `n` grows.

23.1 Diagonal matrix storage

A diagonal matrix stores only `n` values.

So storage is:

`O(n)`

A normal matrix stores:

`n * n = n2 values`

So normal matrix storage is:

`O(n2)`

The diagonal matrix saves memory.

23.2 `create()` complexity

In the diagonal version shown here, `create()` reads only `n` diagonal elements.

So:

`create() = O(n)`

But if a version reads all `n × n` visible positions and stores only useful values, then:

`create() = O(n2)`

Important distinction:

Create style	Time complexity
Read only stored values	$O(n)$ for diagonal matrix.
Read the full visible matrix	$O(n^2)$.

23.3 `get()` complexity

`get()` checks one condition and possibly reads one array value.

So:

```
get() = O(1)
```

$O(1)$ means constant time.

The time does not grow with `n`.

23.4 `set()` complexity

`set()` checks one condition and possibly updates one array value.

So:

```
set() = O(1)
```

23.5 `display()` complexity

`display()` prints the full visible matrix.

A visible matrix has:

```
n × n
```

positions.

So:

```
display() = O(n²)
```

Even though storage is small, display must still print every visible position.

23.6 Complexity summary

For the diagonal modular program:

Operation	Time	Space	Why
Storage	—	<code>O(n)</code>	Store only diagonal values.
<code>create()</code>	<code>O(n)</code>	—	Reads only <code>n</code> diagonal values.
<code>get()</code>	<code>O(1)</code>	—	One condition and one array access.
<code>set()</code>	<code>O(1)</code>	—	One condition and one assignment.
<code>display()</code>	<code>O(n²)</code>	—	Prints all visible matrix positions.

24. Common beginner mistakes

Mistake 1: Putting all logic inside `main()`

This makes the program long and harder to understand.

Better:

```
case 4:
    display(A, n);
    break;
```

Mistake 2: Forgetting function prototypes

If function definitions are written after `main()`, prototypes should be written before `main()`.

Example:

```
void display(int *A, int n);
```

Mistake 3: Using `delete` instead of `delete[]`

Wrong:

```
delete A;
```

Correct:

```
delete[] A;
```

Mistake 4: Storing non-diagonal values in a diagonal matrix

Wrong idea:

```
set(A, 1, 3, 99);
```

This should not store `99` because `(1, 3)` is not diagonal.

Mistake 5: Forgetting the `i - 1` conversion

Matrix positions start at `1`, but array indexes start at `0`.

So:

```
M[3][3] -> A[2]
```

not:

```
M[3][3] -> A[3]
```

Mistake 6: Thinking display is `O(n)`

The stored array has only `n` values, but display prints `n2` positions.

So display is:

```
O(n2)
```

25. Main comparison: Lesson 13 vs Lesson 14

Lesson 13	Lesson 14
Menu-driven program.	Modular menu-driven program.
Logic may be directly inside menu cases.	Logic is moved into helper functions.
<code>switch</code> cases can become long.	<code>switch</code> cases stay short.

Lesson 13

Lesson 14

Harder to modify.

Easier to modify.

Focus is interaction.

Focus is organization.

Lesson 13 answers:

How do we let the user choose operations?

Lesson 14 answers:

How do we organize those operations cleanly?

26. Key ideas to remember

1. Lesson 14 is about modular programming.
2. A modular program is split into smaller functions.
3. `main()` should control the menu.
4. Helper functions should perform the matrix operations.
5. The main helper functions are `create()`, `get()`, `set()`, and `display()`.
6. Function prototypes are written before `main()`.
7. `int *A` is passed to functions so they can use the same array.
8. `set()` can permanently modify the array because it receives the array address.
9. For a diagonal matrix, the stored condition is `i == j`.
10. For a diagonal matrix, the index formula is `i - 1`.
11. Non-diagonal positions are structural zeros.
12. `get()` returns `0` for structural zeros.
13. `display()` prints fake zeros for non-stored positions.
14. Dynamic memory created with `new[]` must be released with `delete[]`.
15. The same modular design can be reused for other special matrices by changing formulas.

27. Exit criteria for Lesson 14

You understand Lesson 14 if you can take a long menu case like this:

```
case 4:
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (i == j) {
                cout << A[i - 1] << " ";
```

```
        } else {  
            cout << "0 ";  
        }  
    }  
    cout << endl;  
}  
break;
```

and rewrite it like this:

```
case 4:  
    display(A, n);  
    break;
```

That is the main goal of Lesson 14.

The menu should call helper functions.

The helper functions should contain the actual matrix logic.

28. Final simple summary

Lesson 14 teaches how to clean up the menu-driven matrix program.

Instead of putting all code inside `main()`, we split the work into functions:

```
create(A, n);  
get(A, i, j);  
set(A, i, j, x);  
display(A, n);
```

The menu becomes short.

The program becomes easier to read.

The matrix logic becomes easier to test.

The same structure can later be reused for diagonal, lower triangular, upper triangular, symmetric, tridiagonal, and Toeplitz matrix programs.

The important habit is this:

Keep the menu in `main()`.
Keep the matrix operations in helper functions.

That is the core idea of a modular menu-driven matrix program.